



THE EDUCATOR · CPCS 351

Topic 01

Introduction to Software Engineering

A complete, study-ready edition of the live Topic 01 content - with the original repository visuals and one consistent design system.

Nature of Software	Need for Engineering	Teamwork & Scale
Life-Cycle Activities	SQA + Project Management	OOSE Specialization

8 lessons · full lesson content · activities · self-feedback · visual anchors

Golden Master · v1.0

Topic 01

Contents

Eight lessons forming one causal story: from the nature of software to software engineering, teamwork, life-cycle activities, quality/project control, and object-oriented specialization.

The Nature of Software	3
What Is Software Engineering?	5
Why Software Engineering?	6
Engineering Software as a Team	7
Software Life Cycle - The Big Picture	9
Software Development Process & the Waterfall Example	11
Software Quality Assurance & Project Management	13
Object-Oriented Software Engineering & Topic Review	14

Golden Master rule

This PDF preserves the instructional content of the live Topic 01 pages. The two large image-based visual anchors are rendered from the original SVG source assets stored in the course repository; the remaining visual anchors reproduce the page's native HTML/CSS concepts in print form.

TOPIC 01 · LESSON 01

1

of 8 lessons

The Nature of Software

Before asking how to engineer software, understand why software behaves differently from physical products.

Start with the puzzle

If software has no factory, no raw material, and does not physically wear out, where do its cost and difficulty come from?

Learning Focus

Explain at least four distinctive properties of software and connect each property to an engineering consequence.

PRACTICE LAB · TOPIC 01

Mission 01 - From Idea to Engineered Software

Turn a Smart Campus idea into an engineered software system through eight diagnostic stages covering software nature, PQCT, teamwork, lifecycle evidence and engineering review.

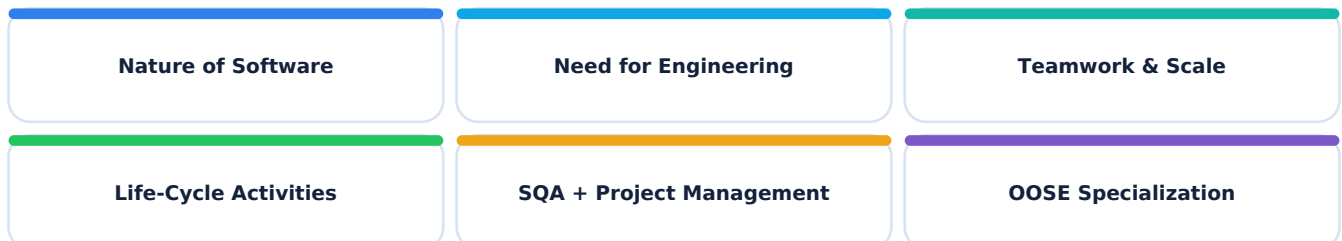
8 Stages · Diagnostic Feedback · No Score-Only Quiz

+

Idea → evidence → engineering review

TOPIC 01 · MASTER CONCEPT MAP

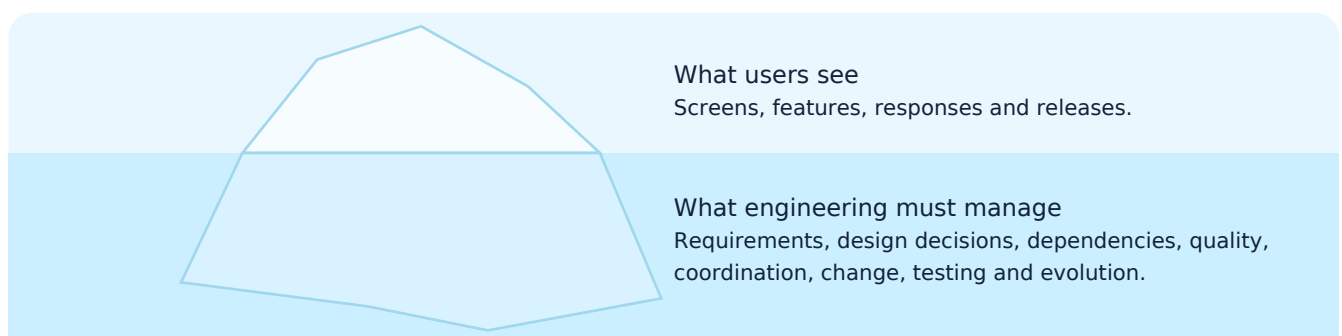
The chapter is one causal story



Read it left to right: software's unusual nature creates engineering difficulty; scale requires coordinated teams; teams need a life-cycle process; development must be supported by quality assurance and project management; OOSE is one specialization for engineering systems around interacting objects.

VISUAL ANCHOR

The Software Iceberg



MUST KNOW

Why Software Is Different

- Software is intangible. Because it cannot be physically inspected like a bridge or machine, development effort and progress are harder to understand.
- Software is easy to reproduce. Once developed, copying it is cheap; most cost lies in development rather than manufacturing.
- The industry is labor-intensive. Much of the intellectual work is difficult to automate completely.
- Untrained people can hack something together. A program may appear to work while serious quality problems remain hidden.
- Software is easy to modify. This encourages changes, sometimes without understanding their wider consequences.
- Software does not physically “wear out.” It deteriorates when its design is changed incorrectly or in unanticipated ways, increasing complexity.

OFFICIAL IDEA, EXPLAINED

Copying Is Cheap; Creating Is Not

Manufacturing another physical product usually consumes material and factory effort. For software, making another copy is nearly free. The expensive part is discovering what to build, designing it, implementing it, checking it, coordinating people, and changing it safely.

Why “easy to modify” can become dangerous

Ease of editing does not mean ease of understanding. A small local change can violate assumptions elsewhere and gradually make the design harder to maintain.

Apply it · Banking App

A bank adds one apparently small rule: international transfers above a threshold require an additional approval. Why can a “small” software change still require requirements analysis, design review, coding, testing, security review and coordinated release?

Reveal reasoning: software is easy to edit but interconnected. The rule can affect UI, business logic, authorization, audit records, tests and operational procedures. Cheap modification at the code level does not mean cheap or safe system change.

Remember

Software's unusual economics and changeability are exactly why disciplined engineering is needed.

TOPIC 01 · LESSON 02

2
of 8 lessons

What Is Software Engineering?

Software engineering applies engineering processes and methods to improve the way software is produced and the quality of what is delivered.

Start with the puzzle

If coding faster reduces time but damages quality, did the project really improve?

Learning Focus

Define software engineering using the chapter terminology and explain the four PQCT goals and their trade-offs.

OFFICIAL DEFINITION

Software Engineering as a Discipline

Software engineering focuses on the research, education, and application of engineering processes and methods to significantly increase software productivity and quality while reducing software cost and time to market.

Visual Anchor · PQCT Dashboard



The goal is not to optimize one number blindly. Engineering decisions balance all four pressures.

WORKED EXAMPLE

A Release Trade-off

A team can shorten time to market by skipping reviews and tests, but that may lower quality and increase later cost. Software engineering asks for a repeatable process that improves the overall PQCT outcome rather than one metric in isolation.

Make the Trade-off Visible

A manager says, “We improved productivity because the team wrote twice as many lines of code.” What is missing from this claim?

Self-feedback: Lines of code alone say little about delivered value. Ask what happened to quality, cost, time to market, and useful functionality.

TOPIC 01 · LESSON 03

3
of 8 lessons

Why Software Engineering?

As software spreads, grows, and becomes expensive and critical, “just code it” stops being a viable development strategy.

Start with the puzzle

Could one engineer build a 500,000-line system alone in a reasonable time?

Learning Focus

Justify the need for software engineering using the chapter's arguments about societal dependence, cost, complexity, embedded systems, scale, and teamwork.

MUST KNOW

Why an Engineering Approach Is Needed

- Software is expanding into all sectors of society; companies rely on it to run and expand their businesses.
- Systems are becoming larger and more complex, reaching millions of lines of code.
- The chapter notes that software can represent 90–95% of total system costs, compared with 5–10% two decades earlier.
- Embedded systems may contain costly application-specific integrated circuits (ASICs), making software quality critical.

Visual Anchor · The Scale Argument

10,000 LOC

100–200 engineer-days

≈ 5–10 months for one engineer



500,000 LOC

5,000–10,000 engineer-days

≈ 20–40 years for one engineer

Conclusion: real-world systems require many software engineers working together.

OFFICIAL EVIDENCE

50–100 Lines per Day Is Not “Slow Typing”

The chapter uses an average of 50–100 source lines per day to make a larger point: engineering time includes understanding, design, coordination, testing, correction and integration—not merely typing code.

Estimate Before You Memorize

If a system is fifty times larger, why can't we simply expect one engineer to finish it fifty times faster by using better typing tools?

Self-feedback: Scale adds intellectual complexity and forces teamwork. Teamwork then introduces conceptualization, communication and coordination problems—the subject of the next lesson.

TOPIC 01 · LESSON 04

4
of 8 lessons

Engineering Software as a Team

Large systems force people to work together. The next engineering problem is making sure they understand, communicate, and coordinate around the same system.

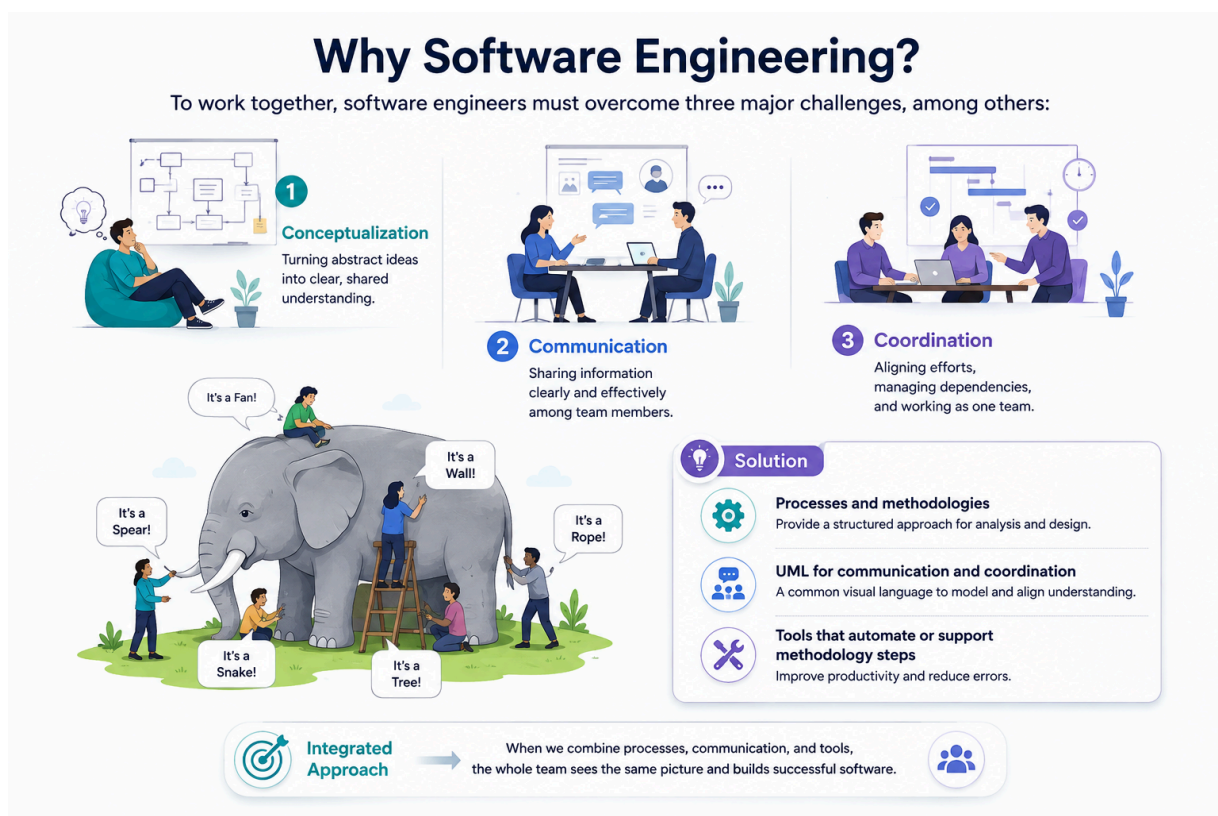
Start with the puzzle

Four engineers can all be correct about their own piece—and still build the wrong whole.

Learning Focus

Explain the three teamwork challenges—conceptualization, communication and coordination—and match each with the software-engineering mechanisms proposed in the chapter.

Visual Anchor · Why Software Engineering?



Large study visual for the official teamwork challenges and engineering response.

OFFICIAL LECTURE STRUCTURE

Three Challenges

Conceptualization

Develop a coherent shared understanding of what the system is and how it should work.

Communication

Express ideas precisely enough that others interpret them consistently.

Coordination

Align interdependent work so components, people and decisions fit together.

The Engineering Response

- Processes and methodologies support analysis and design.
- UML provides a shared visual language for communication and coordination.
- Tools automate or support methodology steps.

Interpret the Elephant

A database engineer says “the system is the data model,” while a UI engineer says “the system is the user journey.” What is the problem?

Self-feedback: Neither view is necessarily false; each is partial. The conceptualization challenge is to create a shared model of the whole, then communicate and coordinate decisions across views.

TOPIC 01 · LESSON 05

5
of 8 lessons

Software Life Cycle - The Big Picture

Software engineering is not one chain of coding steps. The chapter organizes it as three interacting life-cycle tracks.

Start with the puzzle

While developers design and code, who protects quality—and who keeps the project feasible, scheduled and controlled?

Learning Focus

Name the three life-cycle tracks, explain their roles, and show how together they pursue productivity, quality, cost and time-to-market goals.

Visual Anchor · Software Life Cycle Activities



Read the Visual as Three Interacting Tracks

Software Development Process

Transforms the initial concept into an operational system.

Software Quality Assurance

Checks that activities are performed properly and artifacts meet requirements and quality standards.

Software Project Management

Controls and administers development and SQA work.

Important: these are interacting tracks—not Development → then SQA → then Project Management.

Connect the Tracks to PQCT

Development creates the product, SQA provides confidence in process and artifacts, and project management controls effort, schedule, risk and administration. Together they aim to improve Productivity and Quality while controlling Cost and Time to Market.

Map the Responsibility

Classify: designing an architecture; reviewing a requirements document; estimating effort and scheduling a release.

Self-feedback: Development → architecture. SQA → review for conformance/quality. Project Management → estimation and scheduling.

TOPIC 01 · LESSON 06

6

of 8 lessons

Software Development Process & the Waterfall Example

Follow development from the initial system concept to an operational system—and connect every phase to the artifact or evidence it produces.

Start with the puzzle

What has to happen between “we need a system” and “the system is running in its target environment”?

Learning Focus

Explain how the development process transforms a concept into an operational system, correctly order the Waterfall phases, and identify representative deliverables/evidence produced along the way.

DEVELOPMENT PROCESS

What the Process Must Do

A software development process transforms the initial system concept into the operational system running in the target environment. It identifies business needs, conducts a feasibility study, formulates requirements/capabilities, and then designs, implements, tests and deploys the system.

Visual Anchor · Waterfall Cascade

1 · System Engineering

2 · Software Requirements Analysis

3 · Software Design

4 · Coding & Unit Testing

5 · Integration & Integration Testing

6 · Acceptance Testing

7 · Maintenance

CLO COVERAGE · PHASE → EVIDENCE

What Each Phase Produces

The lecture names the phases; the examples below make the phase outputs concrete. Exact document names vary by organization, but each phase should leave evidence that supports the next.

1 · System Engineering

System concept, business need, scope/context and feasibility evidence.

2 · Requirements Analysis

Software Requirements Specification (SRS) or equivalent agreed requirements.

3 · Software Design

Architecture and detailed design models/specifications.

4 · Coding & Unit Testing

Source code, executable units and unit-test results.

5 · Integration & Testing

Integrated build plus integration-test results and defect evidence.

6 · Acceptance Testing

Acceptance results showing whether the delivered system satisfies agreed needs.

7 · Maintenance

Change requests, fixes, revised artifacts and updated releases.

WORKED EXAMPLE · UNIVERSITY REGISTRATION SYSTEM

One System Through the Whole Life Cycle

Need

Students need online registration

SRS

Rules, courses, prerequisites, capacity

Design

UI, services, database, interfaces

Code

Registration modules + unit tests

Integrate

Connect SIS, authentication, billing

Accept

Registrar/students validate scenarios

Maintain

Fix defects and adapt policies

This example is explanatory enrichment: it applies the lecture's phase sequence to one coherent system so students can see how information and artifacts flow forward.

HOW TO READ IT

Waterfall Is an Organization of Development Work

The model makes the progression visible: understand the system context and requirements before design; design before implementation; test units and integrations; validate through acceptance testing; then maintain the delivered system. Later topics can critique or compare lifecycle models, but here the priority is understanding the sequence and purpose of these phases.

Put the Phases Back in Order

Design, Acceptance Testing, Requirements Analysis, Coding & Unit Testing. Which comes first, and what evidence should flow forward?

Self-feedback: Requirements Analysis → Design → Coding & Unit Testing → Acceptance Testing. Requirements guide design; design guides implementation; implementation produces executable software; acceptance provides evidence against agreed needs.

TOPIC 01 · LESSON 07

7

of 8 lessons

Software Quality Assurance & Project Management

Development builds the system. SQA asks whether the work and artifacts meet expectations; project management asks whether the work is controlled and deliverable.

Start with the puzzle

A project can produce high-quality code and still miss the deadline. It can also finish on time and deliver poor quality. Who watches each problem?

Learning Focus

Distinguish SQA from project management and identify the responsibilities listed for each track in the chapter.

Visual Anchor · Quality Gate vs Control Tower

Quality Gate · SQA

Ensures development activities are performed properly and that software artifacts meet software requirements and desired quality standards.

Control Tower · Project Management

Oversees the control and administration of development and SQA activities so delivery remains manageable.

MUST KNOW

Project Management Activities in the Chapter

- Effort estimation
- Project planning and scheduling
- Risk management
- Project administration
- Other control and coordination activities needed to deliver on time and within budget

Spot the Responsibility

“The requirements review found an ambiguous rule.” “The release is two weeks behind schedule.” Which track primarily addresses each?

Self-feedback: The review finding is primarily SQA; schedule control is primarily project management. The tracks interact, but their purposes are different.

Remember

SQA protects process/artifact quality; project management controls the project. Neither replaces development.

TOPIC 01 · LESSON 08

8
of 8 lessons

Object-Oriented Software Engineering & Topic Review

Close the chapter by placing object orientation inside software engineering—and connect every major idea into one mental model.

Start with the puzzle

If the world is made of interacting things with responsibilities, can software models mirror that structure?

Learning Focus

Define OOSE as a specialization of software engineering, identify what it encompasses, and reconstruct the Topic 01 concept chain without notes.

OFFICIAL LECTURE CONTENT

Object-Oriented Software Engineering

OOSE is a specialization of software engineering. The object-oriented paradigm views the world and systems as consisting of objects that relate and interact with each other.

OOSE encompasses:

- OO processes
- OO methodologies
- OO modeling languages
- OO tools

Visual Anchor · An Interacting Object Universe



The point is not these particular objects; it is the paradigm: systems are modeled through related objects and their interactions.

TOPIC 01 · KEY TAKEAWAYS

Rebuild the Story

1. Software is intangible, cheap to reproduce, labor-intensive, easy to modify, and does not physically wear out.

2. Software engineering applies engineering processes and methods to improve P and Q while reducing C and T.

3. Scale makes teamwork unavoidable; teamwork creates conceptualization, communication and coordination challenges.

4. Software engineering operates through three interacting tracks: Development, SQA and Project Management.

5. Development transforms concept into an operational system; Waterfall is one structured sequence of development phases.

6. OOSE specializes software engineering around objects, relationships and interactions.

WHOLE-TOPIC TRANSFER CHALLENGE

Engineer a University Registration System

Do not answer by recalling isolated definitions. Use the chapter as one engineering story.

Nature

Why is the software cheap to copy but expensive to create and evolve safely?

SE trade-off

What would P, Q, C and T mean for this project?

Teamwork

Why can one programmer no longer understand and coordinate the whole product alone?

Life cycle

What activities transform the registration idea into an operational system?

SQA + PM

How do quality assurance and project management support development rather than replace it?

OOSE

What candidate interacting objects might appear in an OO model?

Reveal a model reasoning path

The product is inexpensive to reproduce but costly to understand, coordinate, test and evolve. Engineering balances performance/quality against cost/time. A team is needed because requirements, UI, data, security, testing and deployment create specialization and coordination needs. Development moves through requirements, design, implementation, integration/testing and operation/evolution; SQA provides quality evidence while PM coordinates scope, people, schedule and resources. An OOSE view might model Student, Course, Section, Registration, Instructor and Payment as related objects with responsibilities.

EXAM LENS

Distinctions worth being able to explain

Program vs. engineered software

Working code alone is not evidence that requirements, quality, maintainability, coordination and evolution have been engineered.

Life cycle vs. Waterfall

The life cycle is the broader set of activities over software's existence; Waterfall is one way of organizing development phases.

Development vs. SQA vs. PM

They are interacting tracks: build the product, provide quality confidence, and coordinate the work.

SE vs. OOSE

OOSE does not replace software engineering; it specializes it using the object-oriented paradigm.